

Constructors and Destructors

What is a Constructor?

A **constructor** is a special member function whose purpose is to **initialise the data members of an object** when it is created. It is called automatically when an object is declared.

A constructor has the same name as the class and has **no return type** (not even void).

```
class Item {
    int code;
    float price;
public:
    Item() {                // constructor – runs automatically on object creation
        code = 0;
        price = 0.0;
    }
};

int main() {
    Item i;    // constructor called automatically here
}
```

Characteristics of Constructors

- Must be declared in the **public** section of the class
- Invoked **automatically** when an object is created
- Have **no return type**
- Cannot be **virtual**
- Can have **default arguments**
- Are **not inherited** (but a derived class can call a base class constructor)

Types of Constructors

1. Default Constructor

A constructor that takes **no parameters**. If you don't define any constructor, C++ provides a default one automatically.

```
class Point {
    int x, y;
public:
    Point() {                // default constructor
        x = 0;
        y = 0;
    }
};
```

2. Parameterised Constructor

A constructor that accepts **parameters**, allowing objects to be initialised with specific values.

```
class Point {
    int x, y;
public:
    Point(int a, int b) {    // parameterised constructor
        x = a;
        y = b;
    }
};

int main() {
    Point p1(3, 4);          // implicit call
    Point p2 = Point(5, 6);  // explicit call
}
```

3. Copy Constructor

Used to **initialise a new object as a copy** of an existing object. It takes a reference to an object of the same class as its argument.

```
class Sample {
    int n;
public:
    Sample(int a) { n = a; }
    Sample(Sample &x) { n = x.n; }    // copy constructor
};

int main() {
    Sample A(100);
    Sample B(A);          // copy constructor called
    Sample C = A;         // copy constructor called
}
```

Multiple Constructors (Constructor Overloading)

A class can have **multiple constructors** with different parameter lists. The correct one is chosen automatically based on the arguments provided when creating the object.

```
class Complex {
    float real, img;
public:
    Complex() { real = img = 0; }           // default
    Complex(float r) { real = img = r; }    // single parameter
    Complex(float r, float i) { real=r; img=i; } // two parameters
};
```

What is a Destructor?

A **destructor** is a special member function that is called automatically when an object goes out of scope or is explicitly deleted. Its job is to release any resources the object was using.

A destructor has the same name as the class, preceded by a **tilde (~)**, and takes no arguments and has no return type.

```
class Marks {
public:
    Marks() {
        cout << "Object created" << endl;
    }
    ~Marks() {
        cout << "Object destroyed" << endl;
    }
};

int main() {
    Marks m1;    // constructor called
    Marks m2;    // constructor called
}               // destructors called automatically here – in reverse order
```

Key Differences: Constructor vs Destructor

Feature	Constructor	Destructor
Purpose	Initialise an object	Clean up an object
Called	When object is created	When object goes out of scope
Arguments	Can have parameters	Cannot have parameters
Return type	None	None
Number per class	Multiple (overloaded)	Only one
Prefix	None	~ (tilde)